Instituto Tecnológico de Aeronáutica

FAROL – Descrição do Controle de Configuração

Equipe FAROL

Eduardo Henrique Camargo de Toledo
Matheus Felipe Ramos Borges
Pietro Maragno Trindade Marcucci
Victor Alves
Yuri Barros Mendes

Fundamentos de Engenharia de Software (CSI-28)

Laboratório 04 – Descrição do Controle de Configuração

Profs. Karla Donato Fook e Maylon Pires Macedo

Divisão de Ensino de Engenharia da Computação (IEC)
Instituto Tecnológico de Aeronáutica – ITA

Outubro de 2026

Histórico de revisões

Versão	Data	Autor	Descrição
1.0	15/10/2026	Victor Alves	Versão inicial do Plano de Controle de Configuração (Laboratório 04).

1 Introdução

Este documento descreve como a Equipe FAROL realiza o controle de configuração do FAROL – *Ferramenta de Acompanhamento, Retenção e Orientação Longitudinal*, aplicação web de apoio à decisão para a Direção do Curso Alberto Santos Dumont (CASD). Atendendo ao Laboratório 04, ele (a) informa as ferramentas utilizadas (Seção 3) e (b) descreve como o controle é realizado (Seções 2 e 4 a 7).

A equipe adota a definição ampla de configuração vista em aula [2, 1]: além do código, são controlados requisitos, documentação de arquitetura, executáveis e bibliotecas, processos, ferramentas e infraestrutura.

O projeto está no fim da modelagem: proposta (Lab 01), Documento de Requisitos v1.0 (Lab 02) e Relatório Descritivo (Lab 03) já foram produzidos, mas ainda não há código. Por isso, o controle tem duas frentes: a **configuração documental**, já em operação no Google Drive, e a **configuração do software**, a ser implantada na fase de Prototipagem com Git, GitHub e integração contínua. Cada prática é marcada como **Em uso** (já praticada), **Adotado** (vale a partir desta entrega) ou **Planejado** (fase posterior do cronograma).

2 Itens de Configuração

Itens de Configuração (ICs) são os artefatos cuja evolução é controlada formalmente, com identificador, responsável, local e versão (Tabela 1).

As planilhas reais (IC-04) ficam em pasta restrita e **nunca** vão para o repositório; desenvolvimento e testes usam só dados sintéticos (NF017, LGPD). Para as regras de alerta (IC-10), o repositório guarda apenas os valores padrão, pois o Diretor as altera pelo próprio sistema, que registra autor, data e valor anterior (NF009).

Nomenclatura. Documentos seguem `FAROL_<Artefato>_v<maior>.<menor>.pdf` (o número maior muda com alteração de escopo aprovada; o menor, com correções) e começam com histórico de revisões. Quando a disciplina exige outro nome, como `FAROL_GConfig.pdf`, ele é respeitado. Diagramas guardam o arquivo-fonte ao lado da imagem exportada, com o mesmo nome base. O software usa versionamento semântico [4] por *tags*.

Linhas de base. São configurações aprovadas que só mudam pelo processo da Seção 5. Coincidem com os marcos do cronograma (Tabela 2).

3 Ferramentas utilizadas

As escolhas priorizam custo zero (o cliente é uma ONG, NF024), ausência de servidor próprio para manter e continuidade entre gestões do CASD (NF007, NF019). O Drive já é usado

Tabela 1: Itens de Configuração do FAROL.

ID	Item	Formato / local	Resp.	Situação
Documentação e gestão				
IC-01	Proposta, Doc. de Requisitos e Relatório Descritivo (Labs 01–03)	.tex + PDF / Drive	Victor, Eduardo	**Em uso**
IC-02	Fontes dos diagramas (arquitetura, classes, DER)	.drawio, .pptx / Drive	Eduardo	**Em uso**
IC-03	Atas de reunião com o cliente	Google Docs / Drive	Victor	**Em uso**
IC-04	Planilhas reais do CASD	XLSX / Drive (restrito)	Victor	**Em uso**
IC-05	Registro de Configuração e Mudanças	Google Sheets / Drive	Victor	**Adotado**
IC-06	Este plano, incluindo a relação de ferramentas	.tex + PDF / Drive	Victor	**Adotado**
Software e infraestrutura				
IC-07	Código-fonte, esquema do banco e migrações	Repositório Git	Matheus, Pietro	**Planejado**
IC-08	Dependências, contêineres e `.env.example`	Repositório Git	Pietro	**Planejado**
IC-09	Pipeline de CI, testes e bases sintéticas	Repositório Git	Yuri	**Planejado**
IC-10	Valores padrão das regras de alerta (RF018) e guia de uso	Repositório Git	Eduardo	**Planejado**
IC-11	Hospedagem, banco de produção e backup (NF024)	Provedor de baixo custo	Victor	**Planejado**

Tabela 2: Linhas de base do projeto.

Linha	Marco	Situação
LB-0 a LB-2	Labs 01, 02 e 03	Estabelecidas
LB-3	Configuração (Lab 04)	Esta entrega
`v0.1.0`	Protótipo validado	Planejada
`v0.2.0`–`v0.4.0`	Ingestão; Indicadores e Alertas; Intervenções e Relatórios	Planejadas
`v1.0.0`	Entrega final validada com o cliente	Planejada

pela equipe e pela Direção. O Git é distribuído e permite trabalho *off-line*, adequado a uma equipe sem horário comum. GitHub Issues e Actions dispensam servidores como os de Redmine, Bugzilla ou Jenkins e ligam cada mudança ao seu commit. O Docker evita o "na minha máquina funciona". O repositório ficará numa *organização* do GitHub, para que possa ser transferido ao CASD ao fim da disciplina.

Tabela 3: Ferramentas de controle de configuração do FAROL.

Finalidade	Ferramenta	Uso no projeto	Situação
Repositório de documentos	Google Drive	Artefatos em pastas padronizadas, com controle de acesso e histórico de revisões	**Em uso**
Documentos colaborativos	Google Docs e Sheets	Atas e Registro de Configuração, com autor e data de cada alteração	**Em uso**
Relatórios e diagramas	LaTeX, Draw.io, PowerPoint	Fontes dos documentos entregues e dos diagramas	**Em uso**
Controle de versões	Git + GitHub	Código e demais ICs de software	**Planejado**
Controle de mudanças	GitHub Issues e Projects	Solicitações de mudança, defeitos e quadro Kanban	**Planejado**
Integração contínua	GitHub Actions	Build, estilo e testes a cada *pull request*	**Planejado**
Ambiente	Docker Compose	Mesmo ambiente em todas as máquinas e servidores	**Planejado**

4 Controle da configuração documental

Os artefatos ficam em uma pasta compartilhada do Drive (Figura 1), que separa material editável de entregas congeladas.

```
FAROL/
  00_Gestao/                          atas e Registro de Configuração
  01_Entregas/                        PDFs submetidos – somente leitura
  02_Fontes/ e 03_Diagramas/          .tex, .drawio, .pptx e imagens
  04_Dados_Cliente/                   planilhas reais – acesso restrito
  05_Referencias/                     enunciados e bibliografia
```

Figura 1: Estrutura de pastas do projeto no Google Drive.

- **Edição:** o histórico de versões do Drive registra autor e data de cada alteração e permite recuperar versões anteriores.
- **Congelamento:** na entrega, o gerente de configuração copia o PDF versionado para `01_Entregas`, onde só ele tem escrita, e marca a revisão para ser mantida permanentemente. Isso estabelece a linha de base.
- **Dados reais:** `04_Dados_Cliente` é compartilhada só com a equipe, nunca por link público, e não é copiada para relatórios ou código.

5 Controle de mudanças

Toda alteração em um IC de linha de base (novo requisito da Direção, correção de regra de negócio, ajuste em diagrama entregue) passa por uma **Solicitação de Mudança (SM)**. Até existir o repositório, as SMs ficam no Registro de Configuração (IC-05); depois, viram *issues* do GitHub. Cada SM registra identificador, solicitante, tipo (corretiva, evolutiva ou de requisito), motivação, ICs e requisitos afetados, análise de impacto, decisão e versão resultante. O fluxo

está na Figura 2; a análise de impacto usa os identificadores RF, RN e NF do Documento de Requisitos.

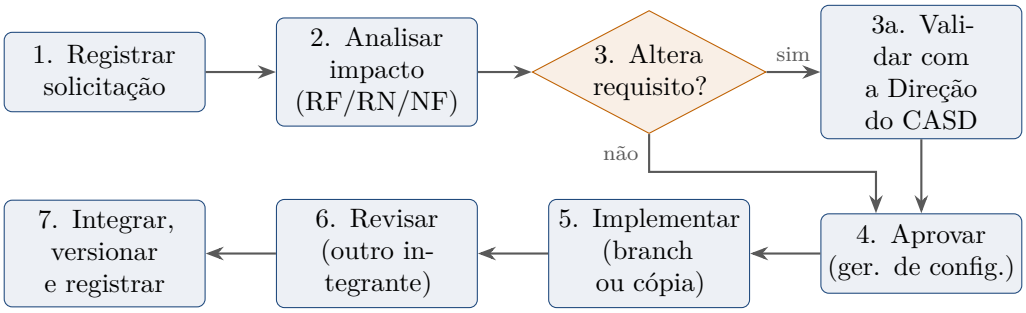


Figura 2: Fluxo de tratamento de uma Solicitação de Mudança.

Mudanças de requisito ou escopo geram nova versão maior do Documento de Requisitos; mudanças de arquitetura ou de dados exigem atualizar o Relatório Descritivo e os diagramas. Quem implementa não revisa, e SMs rejeitadas ficam registradas com a justificativa.

6 Controle de versões e integração contínua

Situação: **Planejado** – implantação no início da Prototipagem, partindo do protótipo herdado do PEI.

Repositório. Um único repositório (monorepo), com diretórios que espelham as camadas da arquitetura do Lab 03 (`ingestao/`, `inteligencia/`, `core/`, `relatorios/`, `web/`, `db/migrations/`, `tests/`, `docs/`). Como uma mudança em um indicador costuma tocar regras, banco e interface, ela cabe em um só *pull request*. O `.gitignore` exclui `.env` e planilhas fora de `tests/dados_sinteticos/`, impedindo que dados reais ou credenciais entrem no histórico.

Branches e commits. Adota-se Trunk-Based Development [3]: a `main` é protegida e só recebe *pull requests* aprovados por outro integrante e com a CI aprovada. Cada tarefa gera uma *branch* curta (ex.: `feat/RF003-importar-frequencia`), integrada em até uma semana. Os commits seguem Conventional Commits [5] e citam o requisito, o que fecha a rastreabilidade *requisito* → *issue* → *commit* → *pull request* → *tag*.

Integração contínua. A cada *pull request*, o GitHub Actions monta o ambiente, verifica o estilo, aplica as migrações em um banco vazio e roda os testes, priorizando as regras de negócio dos alertas (RN001, RN002, RN005, RN008, RN009) com dados sintéticos. Qualquer falha bloqueia a integração.

Ambientes. *Desenvolvimento* e *homologação* usam dados sintéticos; a homologação é atualizada a cada integração na `main` e serve para validar com a Direção. A *produção* usa dados reais e só recebe versões marcadas por *tag*, liberadas manualmente pelo gerente de configuração, pois contém dados de alunos, alguns menores de idade. Credenciais ficam em `.env` não versionado e nos *secrets* do GitHub; o banco evolui só por migrações e tem backup periódico (NF024).

7 Papéis e auditoria

Ao fechar cada linha de base, o engenheiro de testes verifica de forma independente se os ICs previstos existem na versão e no local declarados, se o PDF entregue corresponde à fonte, se as

Tabela 4: Responsabilidades no controle de configuração.

Integrante	**Papel**	**Responsabilidades**
Victor Alves	Gerente de configuração	Mantém o Registro; aprova SMs; estabelece linhas de base e *tags*; administra acessos; libera versões para produção
Eduardo H. C. de Toledo	Analista de sistemas	Análise de impacto das SMs; rastreabilidade; atualiza requisitos, relatório e diagramas
Matheus F. R. Borges	Programador	Implementa SMs no código e no banco; revisa PRs de Pietro
Pietro M. T. Marcucci	Programador	Implementa SMs na ingestão; mantém dependências e contêineres; revisa PRs de Matheus
Yuri B. Mendes	Engenheiro de testes	Mantém CI e dados sintéticos; revisa testes; conduz as auditorias
Direção do CASD	Cliente	Aprova mudanças de requisito e de escopo

SMs aprovadas foram incorporadas, se não há dados reais fora da pasta restrita e, a partir da `v0.1.0`, se a *tag* aponta para um commit da `main` com CI aprovada. O resultado é anotado no Registro de Configuração (IC-05).

O quadro Kanban do Lab 03 será criado diretamente no GitHub Projects junto com o repositório, para que cada cartão corresponda a uma *issue* rastreável. Até lá, as tarefas são acompanhadas nas reuniões e no Registro de Configuração.

Referências

[1] PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de Software: uma abordagem profissional*. 9. ed. Porto Alegre: AMGH, 2021.

[2] MACEDO, M.; FOOK, K. D. *Gerência de configuração* e *Ferramentas CASE para Gerenciamento de Configuração*. Notas de aula, CSI-28. São José dos Campos: ITA, 2026.

[3] VALENTE, M. T. *Engenharia de Software Moderna*. Belo Horizonte: edição do autor, 2020.

[4] PRESTON-WERNER, T. *Versionamento Semântico 2.0.0*. Disponível em: `https://semver.org/lang/pt-BR/`.

[5] CONVENTIONAL COMMITS. *Conventional Commits 1.0.0*. Disponível em: `https://www.conventionalcommits.org/pt-br/v1.0.0/`.